

04_DEVOPS_ARCHITECTURE

- [SUOP ERP v1.0 — Enterprise DevOps Architecture](#)
 - [1. Purpose](#)
 - [2. Environments](#)
 - [3. Docker](#)
 - [4. CI/CD](#)
 - [5. Git Strategy](#)
 - [6. Deployment](#)
 - [7. Secrets Management](#)
 - [8. Infrastructure as Code](#)
 - [9. Monitoring](#)
 - [10. Logging](#)
 - [11. Tracing](#)
 - [12. Alerting](#)
 - [13. Environment Configuration](#)
 - [14. Backup & Recovery](#)
 - [15. Open Questions for Approval](#)
 - [Approval Block](#)

SUOP ERP v1.0 — Enterprise DevOps Architecture

Field	Value
Document Version	1.0
Status	DRAFT — Awaiting Approval
Date	2026-07-11
Approval Required	Project Owner

1. Purpose

This document defines the complete DevOps architecture for SUOP ERP v1.0: how code moves from developer laptop to production, how environments are managed, how deployments are released and rolled back, and how the system is monitored.

Hard rule: No code reaches production except via this pipeline. No manual SSH-and-edit. No “hotfix directly on the server.” Every change is a PR → CI → CD pipeline.

2. Environments

2.1 Environment Catalog

Environment	Purpose	Data	Access
Local	Developer machine	Synthetic seed data	Developer
CI	Test runner	Test fixtures	CI system only
Staging	Pre-production testing	Anonymized production snapshot	All developers + QA
Production	Live customer use	Real data	Operations team only

2.2 Environment Parity

Staging must match production as closely as possible: - Same Postgres version - Same Redis version - Same Node/Bun version - Same environment variables (different values) - Same infrastructure (smaller instance sizes for cost) - Same third-party integrations (test accounts)

Divergences allowed: - Instance size (staging smaller) - Backup frequency (staging less frequent) - Number of replicas (staging single instance) - External integrations use sandbox/test accounts

2.3 Environment Promotion

```
Local → CI (auto on PR) → Staging (auto on merge to develop)
→ Production (manual approval on merge to main)
```

- **Local → CI:** Automatic on PR creation
- **CI → Staging:** Automatic on merge to `develop` branch
- **Staging → Production:** Manual approval gate after merge to `main`; deploy during business hours (10 AM - 4 PM IST)

2.4 Production Deploy Windows

- **Standard:** Tuesday + Thursday, 10 AM - 4 PM IST
 - **Emergency hotfix:** Anytime, requires SEV-1/SEV-2 + on-call approval
 - **Blackout periods:** Quarter-end (GST filing), audit periods, festival peaks (Diwali, etc.)
 - **No Friday deploys** (prevents weekend incident response)
-

3. Docker

3.1 Container Strategy

All three apps (web, backend, mobile-API) ship as Docker containers: - **Reproducible** — same image runs in staging and production - **Isolated** — dependencies bundled - **Scalable** — orchestrate via Kubernetes or container service - **Secured** — non-root user, read-only filesystem, scanned images

3.2 Dockerfile — Backend (docker/Dockerfile.backend)

```
# Multi-stage build for smallest image
FROM oven/bun:1.1-alpine AS base
WORKDIR /app

# Stage 1: Install dependencies
FROM base AS deps
COPY package.json bun.lockb ./
COPY apps/backend/package.json ./apps/backend/
COPY packages/shared/package.json ./packages/shared/
COPY packages/prisma/package.json ./packages/prisma/
RUN bun install --frozen-lockfile --production

# Stage 2: Build
FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN bun run build:backend

# Stage 3: Production image
FROM oven/bun:1.1-alpine AS runner
WORKDIR /app

# Non-root user
RUN addgroup -g 1001 -S nodejs && adduser -S suop -u 1001
USER suop

# Copy built artifacts
COPY --from=builder --chown=suop:nodejs /app/apps/backend/
  dist ./dist
COPY --from=builder --chown=suop:nodejs /app/apps/backend/
  package.json ./package.json
COPY --from=builder --chown=suop:nodejs /app/node_modules ./
  node_modules
COPY --from=builder --chown=suop:nodejs /app/packages ./
  packages
```

```
# Read-only filesystem
RUN mkdir /tmp/suop && chown suop:nodejs /tmp/suop
ENV TMPDIR=/tmp/suop

# Health check
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -q --spider http://localhost:3030/api/v1/_internal/health || exit 1

EXPOSE 3030
CMD ["bun", "run", "dist/main.js"]
```

3.3 Dockerfile — Web (docker/Dockerfile.web)

```
FROM node:20-alpine AS base
RUN apk add --no-cache libc6-compat
WORKDIR /app

# Stage 1: Install
FROM base AS deps
COPY package.json package-lock.json ./
COPY apps/web/package.json ./apps/web/
COPY packages/shared/package.json ./packages/shared/
COPY packages/ui/package.json ./packages/ui/
RUN npm ci

# Stage 2: Build
FROM base AS builder
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build:web

# Stage 3: Production (Next.js standalone)
FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
```

```
ENV NEXT_TELEMETRY_DISABLED=1

RUN addgroup -g 1001 -S nodejs && adduser -S nextjs -u 1001
USER nextjs

COPY --from=builder --chown=nextjs:nodejs /app/apps/
web/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/apps/
web/.next/static ./apps/web/.next/static
COPY --from=builder --chown=nextjs:nodejs /app/apps/web/
public ./apps/web/public

EXPOSE 3000
ENV PORT=3000
CMD ["node", "apps/web/server.js"]
```

3.4 Docker Compose (Development)

docker/docker-compose.yml :

```
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: suop
      POSTGRES_USER: suop
      POSTGRES_PASSWORD: suop_dev_only
    ports: ['5432:5432']
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U suop']
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
```

```
ports: ['6379:6379']
command: redis-server --requirepass suop_dev_only

minio:
  image: minio/minio
  ports: ['9000:9000', '9001:9001']
  environment:
    MINIO_ROOT_USER: suop
    MINIO_ROOT_PASSWORD: suop_dev_only
  command: server /data --console-address ":9001"
  volumes:
    - miniodata:/data

mailhog:
  image: mailhog/mailhog
  ports: ['1025:1025', '8025:8025']

backend:
  build:
    context: ..
    dockerfile: docker/Dockerfile.backend
  ports: ['3030:3030']
  environment:
    DATABASE_URL: postgresql://
      suop:suop_dev_only@postgres:5432/suop
    REDIS_URL: redis://:suop_dev_only@redis:6379
    # ... (other env vars)
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_started
  volumes:
    - ../apps/backend/src:/app/apps/backend/src # hot
      reload in dev
  command: bun run dev

web:
```

```
build:
  context: ..
  dockerfile: docker/Dockerfile.web
ports: ['3000:3000']
environment:
  NEXT_PUBLIC_API_URL: http://localhost:3030
depends_on:
  - backend

volumes:
  pgdata:
  miniodata:
```

3.5 Image Scanning

- **Trivy** scans every image build in CI
- **High/critical vulnerabilities** block deploy
- **Base images** updated monthly (`npm run update:base-images`)
- **SBOM** (Software Bill of Materials) generated via `syft` and stored with image

4. CI/CD

4.1 Pipeline Overview

PR Created

↓

[CI Pipeline]

- ├─ Lint (ESLint + Prettier)
- ├─ Type Check (tsc --noEmit)
- ├─ Unit Tests (Vitest)
- ├─ Integration Tests (Testcontainers)
- ├─ SAST Security Scan (SonarQube)
- ├─ Dependency Scan (npm audit + Snyk)
- ├─ Secret Scan (git-secrets)


```
├─ Build (Next.js + Bun)
├─ E2E Tests (Playwright – only on staging deploy)
└─ Coverage Gate (≥70% business, ≥90% foundation)
↓
PR Merged to develop
↓
[CD Staging Pipeline]
├─ Build production images
├─ Push to container registry
├─ Run migrations (with rollback SQL tested)
├─ Deploy to staging
├─ Smoke tests
└─ Notify Slack
↓
PR Merged to main
↓
[CD Production Pipeline – Manual Approval]
├─ Build production images (re-use staging images if
same commit)
├─ Push to container registry
├─ Deploy to production (blue-green)
├─ Run migrations (in maintenance window if destructive)
├─ Smoke tests
├─ Notify Slack + email
└─ Rollback if smoke tests fail
```

4.2 GitHub Actions — CI

`.github/workflows/ci.yml`:

```
name: CI
on:
  pull_request:
    branches: [main, develop]

jobs:
  lint:
```

```
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: oven-sh/setup-bun@v1
  - run: bun install --frozen-lockfile
  - run: bun run lint
  - run: bun run typecheck
```

unit-tests:

```
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - uses: oven-sh/setup-bun@v1
  - run: bun install --frozen-lockfile
  - run: bun run test:unit --coverage
  - uses: codecov/codecov-action@v4
  - name: Coverage gate
    run: |
      FOUNDATION_COV=$(bun run scripts/read-coverage.ts
      core)
      BUSINESS_COV=$(bun run scripts/read-coverage.ts
      modules)
      if [ "$FOUNDATION_COV" -lt 90 ]; then exit 1; fi
      if [ "$BUSINESS_COV" -lt 70 ]; then exit 1; fi
```

integration-tests:

```
runs-on: ubuntu-latest
services:
  postgres:
    image: postgres:16-alpine
    env:
      POSTGRES_PASSWORD: test
    ports: ['5432:5432']
    options: >-
      --health-cmd pg_isready
      --health-interval 5s
      --health-retries 5
  redis:
```

```
    image: redis:7-alpine
    ports: ['6379:6379']
steps:
  - uses: actions/checkout@v4
  - uses: oven-sh/setup-bun@v1
  - run: bun install --frozen-lockfile
  - run: bun run test:integration

security-scan:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: snyk/actions/node@master
      with:
        command: test
        args: --severity-threshold=high
    env:
      SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
    - uses: github/codeql-action/init@v3
      with:
        languages: javascript-typescript
    - uses: github/codeql-action/analyze@v3

build:
  needs: [lint, unit-tests, integration-tests, security-
    scan]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: oven-sh/setup-bun@v1
    - run: bun install --frozen-lockfile
    - run: bun run build
    - uses: actions/upload-artifact@v4
      with:
        name: build-artifacts
        path: |
```

```
apps/web/.next
apps/backend/dist
```

4.3 GitHub Actions — CD Staging

`.github/workflows/cd-staging.yml`:

```
name: CD Staging
on:
  push:
    branches: [develop]

jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Build & push images
        run: |
          docker build -f docker/Dockerfile.backend -t suop/
            backend:staging-${{ github.sha }} .
          docker build -f docker/Dockerfile.web -t suop/
            web:staging-${{ github.sha }} .
          docker push suop/backend:staging-${{ github.sha }}
          docker push suop/web:staging-${{ github.sha }}
      - name: Run migrations
        run: bun run prisma migrate deploy
        env:
          DATABASE_URL: ${ secrets.STAGING_DATABASE_URL }
      - name: Deploy to staging
        run: ./scripts/deploy-staging.sh ${ github.sha }
      - name: Smoke tests
        run: bun run test:smoke --env staging
      - name: Notify Slack
        if: always()
        uses: slackapi/slack-github-action@v1
```

```
with:
  slack-message: "Staging deploy ${github.sha}:
${job.status}"
```

4.4 GitHub Actions — CD Production

.github/workflows/cd-production.yml:

```
name: CD Production
on:
  push:
    branches: [main]

jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://app.sudhamrit.com
    steps:
      - uses: actions/checkout@v4
      - name: Manual approval gate
        uses: trstringer/manual-approval@v1
        with:
          approvers: ops-lead,security-lead
          minimum-approvals: 2
          issue-title: "Production deploy: $
${github.sha}"
      - name: Run migrations
        run: bun run prisma migrate deploy
        env:
          DATABASE_URL: ${secrets.PROD_DATABASE_URL}
      - name: Deploy (blue-green)
        run: ./scripts/deploy-production.sh $
${github.sha}
      - name: Smoke tests
        run: bun run test:smoke --env production
      - name: Rollback if smoke fails
```

```
    if: failure()
    run: ./scripts/rollback-production.sh
- name: Notify Slack + email
    if: always()
    run: ./scripts/notify-deploy.sh ${ job.status }
```

4.5 Pipeline Gates

Gate	Trigger on Fail
Lint fails	Block merge
Type check fails	Block merge
Unit tests fail	Block merge
Coverage < threshold	Block merge
Integration tests fail	Block merge
Security scan finds high/critical	Block merge
Build fails	Block merge
Smoke tests fail (staging)	Alert, don't block production (staging only)
Smoke tests fail (production)	Auto-rollback
Manual approval pending	Wait for human

5. Git Strategy

5.1 Branching Model — GitHub Flow + Develop

```
main (production)
  ↑
develop (integration)
```

```
↑
feature/<scope>-<description> (short-lived)
↑
hotfix/<scope>-<description> (branched from main)
```

5.2 Branch Lifetimes

Branch Type	Lifetime	Owner
main	Permanent	Operations
develop	Permanent	Engineering
feature/*	<2 weeks	Feature developer
fix/*	<1 week	Bug fixer
hotfix/*	<24 hours	On-call
release/*	<1 week	Release manager

5.3 Branch Naming

- feature/<scope>-<description> — feature/purchase-order-approval-workflow
- fix/<scope>-<description> — fix/stock-ledger-fefo-edge-case
- hotfix/<scope>-<description> — hotfix/auth-jwt-validation-bypass
- chore/<description> — chore/upgrade-prisma-6.12
- docs/<description> — docs/api-standards-update

5.4 Commit Conventions (Conventional Commits)

```
<type>(<scope>): <subject>
```

```
<body>
```

<footer>

Types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `perf`, `ci`, `build`

Scope: module name (`purchase-order`, `auth`, `core-workflow`)

Examples:

```
feat(purchase-order): implement P0 submission workflow
fix(stock-ledger): correct FEFO batch selection edge case
refactor(core-audit): extract audit logger from middleware
test(iqc): add integration tests for fail disposition
docs(architecture): add Phase 0 foundation document
chore(deps): upgrade Prisma to 6.12
ci(cd-production): add manual approval gate
```

5.5 PR Standards

- **Title:** Conventional commit format
- **Description:** What, Why, How, Testing Done, Screenshots (if UI)
- **Size:** Max 500 lines changed (split larger work)
- **Reviewers:** 1 for code; 2 for security-sensitive (auth, RBAC, crypto)
- **CI:** All checks green
- **No `console.log`** in merged code
- **Squash & merge** to keep history clean

5.6 Protected Branches

- `main` and `develop` are protected
 - No force pushes (`git push --force` blocked)
 - No direct commits (must be via PR)
 - Code owners required for specific paths (`packages/prisma/schema.prisma`, `apps/backend/src/core/`)
-

6. Deployment

6.1 Deployment Strategy — Blue-Green

Production uses blue-green deployment: - Two identical environments: `blue` (current production) and `green` (next production) - Deploy new version to `green` - Run smoke tests on `green` - Switch load balancer from `blue` → `green` - Keep `blue` running for 1 hour (fast rollback) - After 1 hour, decommission old `blue`; promote `green` to `blue` for next deploy

Why blue-green: - Zero-downtime deploys - Instant rollback (switch back to `blue`) - Full smoke test on production-equivalent infra before cutover

6.2 Database Migrations in Deploy

Migrations are **forward-only** during deploy: 1. Run migration on `green` environment DB (separate from `blue`) 2. If migration succeeds → continue deploy 3. If migration fails → `green` rolled back; `blue` untouched

For destructive migrations (drop column, change type): 1. **Phase 1 release:** Add new column, dual-write to old + new 2. **Phase 2 release:** Migrate data, read from new column 3. **Phase 3 release:** Stop writing to old column (still in schema) 4. **Phase 4 release (1+ month later):** Drop old column

6.3 Rollback Strategy

Application rollback: - Switch load balancer from `green` back to `blue` - Instant (< 30 seconds) - No data loss (DB is shared between blue/green)

Database rollback: - Migrations are forward-only — no automatic rollback - Rollback SQL paired with every migration; run manually if needed - For schema changes: rollback migration creates the inverse operation - For data changes: rollback requires data restore from backup (rare; avoided by careful migration design)

6.4 Smoke Tests Post-Deploy

```
// tests/smoke/smoke.test.ts
describe('Post-deploy smoke tests', () => {
  it('responds to health check', async () => {
    const res = await fetch(`${BASE_URL}/api/v1/_internal/health`)
    expect(res.status).toBe(200)
  })
  it('login works', async () => {
    const res = await fetch(`${BASE_URL}/api/v1/auth/login`, { method: 'POST', body: ... })
    expect(res.status).toBe(200)
  })
  it('can list products', async () => {
    const res = await fetch(`${BASE_URL}/api/v1/products`, {
      headers: { Authorization: `Bearer ${token}` } })
    expect(res.status).toBe(200)
    expect((await res.json()).data.length).toBeGreaterThan(0)
  })
  // 20-30 critical-path smoke tests
})
```

- 20-30 tests covering critical paths
- Runs in <2 minutes
- Failure triggers auto-rollback

6.5 Mobile App Deployment

- **Mobile:** React Native + Expo
 - **Build:** EAS Build (Expo Application Services)
 - **Distribution:** EAS Submit to App Store + Play Store
 - **Phased rollout:** 10% → 50% → 100% over 7 days
 - **Rollback:** Cannot rollback mobile apps — must ship new version (Phase 0: forced-update capability)
-

7. Secrets Management

(see Security Architecture §10)

7.1 CI/CD Secrets

- Stored in GitHub Actions secrets
- Injected as environment variables in workflow
- Never logged (auto-redacted by GitHub)
- Scoped to environment (staging secrets vs production secrets)

7.2 Runtime Secrets

- Application reads secrets from secrets manager at boot
- Cached in memory for 5 minutes; refreshed periodically
- Secret rotation does not require app restart (cached value expires; new value fetched)

8. Infrastructure as Code

8.1 Terraform

All infrastructure defined in Terraform: - VPC, subnets, security groups - RDS Postgres instance - ElastiCache Redis - S3 buckets - ECS cluster (or EKS) - Load balancers, target groups - CloudWatch alarms - Route53 DNS records - ACM certificates

8.2 Terraform Layout

```
infra/
├─ modules/
│   ├─ vpc/
│   ├─ rds/
│   ├─ redis/
│   └─ s3/
```

```
|   ├── ecs/
|   └── alb/
├── environments/
|   ├── staging/
|   |   ├── main.tf
|   |   ├── variables.tf
|   |   └── terraform.tfvars
|   └── production/
|       ├── main.tf
|       ├── variables.tf
|       └── terraform.tfvars
└── README.md
```

8.3 Terraform Workflow

1. Edit `.tf` files
2. `terraform plan` locally to preview changes
3. PR with plan output in description
4. CI runs `terraform validate` + `terraform plan`
5. Reviewer approves
6. Merge triggers `terraform apply` in CI (with manual approval for production)

8.4 State Management

- State stored in S3 backend with DynamoDB lock
- State file encrypted (S3 SSE-KMS)
- State contains sensitive outputs (DB endpoint, etc.) — never logged in CI

8.5 Drift Detection

- Daily job runs `terraform plan` against production
 - If drift detected → alert ops
 - Drift caused by manual changes (forbidden) → investigate + revert
-

9. Monitoring

9.1 Monitoring Stack

Layer	Tool	Purpose
Metrics	Prometheus + Grafana	Time-series metrics, dashboards, alerting
Logs	Loki (or ELK)	Structured log aggregation
Traces	Tempo (or Jaeger)	Distributed tracing
Errors	Sentry	Error tracking + stack traces
Uptime	Pingdom / UptimeRobot	External uptime monitoring
Synthetic	Checkly	Synthetic transaction monitoring
Real User	Sentry Performance / Datadog RUM	Frontend performance

9.2 Metrics (Prometheus)

Exported by backend at `GET /api/v1/_internal/metrics`:-

- `http_requests_total{method, route, status}` — request count -
- `http_request_duration_seconds{method, route}` — histogram -
- `http_requests_in_flight` — gauge -
- `db_connections_active`, `db_connections_idle` -
- `db_query_duration_seconds{operation}` — histogram -
- `redis_operations_total{operation, status}` -
- `workflow_transitions_total{entity, from, to}` -
- `audit_log_writes_total` -
- `notification_delivery_total{channel, status}` -
- `job_duration_seconds{job_name}` -

```
event_bus_subscribers_total{event_name} - cache_hits_total,  
cache_misses_total
```

9.3 Dashboards (Grafana)

- **System Overview:** Request rate, error rate, latency, CPU, memory, disk
- **Database:** Connections, query latency, slow queries, lock waits, cache hit ratio
- **Business:** POs created/day, GRNs posted/day, NCRs opened/day, COAs generated/day
- **Per-module:** Each major module has its own dashboard
- **Real-time:** Live request rate, error rate, current in-flight requests

9.4 Alerting Rules

```
# Critical alerts (PageDuty / phone call)  
- alert: HighErrorRate  
  expr: rate(http_requests_total{status=~"5.."}[5m]) /  
        rate(http_requests_total[5m]) > 0.05  
  for: 5m  
  annotations:  
    summary: "Error rate > 5% for 5 min"  
  
- alert: ServiceDown  
  expr: up{job="backend"} == 0  
  for: 1m  
  annotations:  
    summary: "Backend service down"  
  
- alert: DatabaseDown  
  expr: pg_up == 0  
  for: 30s  
  annotations:  
    summary: "PostgreSQL down"  
  
# Warning alerts (Slack)
```

```
- alert: HighLatency
  expr: histogram_quantile(0.95,
    http_request_duration_seconds_bucket) > 1
  for: 10m
  annotations:
    summary: "p95 latency > 1s for 10 min"

- alert: DbConnectionsHigh
  expr: pg_stat_database_numbackends > 80
  for: 5m
  annotations:
    summary: "DB connections > 80"

- alert: DiskSpaceLow
  expr: 100 - (node_filesystem_avail_bytes /
    node_filesystem_size_bytes) * 100 > 80
  for: 5m
  annotations:
    summary: "Disk > 80% full"
```

10. Logging

10.1 Logging Stack

Application (Pino) → stdout → Docker → Fluent Bit → Loki → Grafana

10.2 Log Levels

Level	Use	Volume
<code>fatal</code>	App cannot start / continue	Rare
<code>error</code>	Unhandled errors, 5xx	Moderate

Level	Use	Volume
warn	4xx errors, deprecation warnings, slow ops	Moderate
info	Normal operations (login, CRUD, workflow transitions)	High
debug	Detailed flow (SQL queries, cache hits)	High (dev only)
trace	Function-level tracing	Very high (dev only)

10.3 Structured Logging

Every log line is JSON:

```
{
  "level": "info",
  "time": "2026-07-11T14:30:00.000Z",
  "correlationId": "01HXY...",
  "userId": "01HXY...",
  "tenantId": "01HXY...",
  "msg": "Purchase order created",
  "poId": "01HXY...",
  "poNumber": "PO-2026-000001",
  "supplierId": "01HXY...",
  "totalValue": 12500,
  "durationMs": 145
}
```

10.4 Sensitive Data Redaction

- Fields matching `/password|token|secret|key|auth|credit|aadhaar|pan/i` redacted

- PII fields (email, phone) hashed for correlation without exposure
- Redaction config in `logging/redaction-config.ts`

10.5 Log Retention

- Hot (Loki): 30 days
- Warm (S3): 90 days
- Cold (Glacier): 1 year
- Audit logs: see Data Architecture §13

11. Tracing

11.1 Distributed Tracing

- OpenTelemetry instrumentation on backend
- Every HTTP request creates a span
- DB queries, Redis ops, external calls create child spans
- Spans export to Tempo (or Jaeger)
- Correlation ID propagated via `traceparent` header

11.2 Trace Sampling

- 100% of error traces
- 10% of normal traces (random sampling)
- 100% of slow traces (>1s)

11.3 Use Cases

- “Why is this request slow?” → trace shows which DB query / external call is the bottleneck
 - “Why did this fail?” → trace shows error span with stack trace + context
 - “What’s the impact of this service?” → trace heatmaps show dependencies
-

12. Alerting

12.1 Alert Routing

Severity	Channel	Response Time
Critical	PagerDuty → phone call	5 min
High	PagerDuty → Slack	30 min
Medium	Slack	4 hours
Low	Daily digest email	24 hours

12.2 Alert Definitions

System alerts: - Service down - High error rate (>5% for 5 min) - High latency (p95 > 1s for 10 min) - DB connections high - Disk space low - Memory high - CPU sustained high

Business alerts: - NCR severity CRITICAL created → notify quality head - Recall initiated → notify all stakeholders - CAPA overdue → notify owner + escalation - Stock below reorder → notify procurement - Compliance certificate expiring in 30 days → notify quality

Security alerts: - Failed login attempts spike - Permission denied attempts spike - Cross-tenant access attempts - New device login - API key abuse (unusual usage pattern)

12.3 Alert Fatigue Prevention

- **Grouping:** Similar alerts grouped (e.g., 100 5xx errors in 5 min = 1 alert)
 - **Inhibition:** Critical alert suppresses warning alerts for same service
 - **Silencing:** Maintenance windows silence non-critical alerts
 - **Auto-resolution:** Alert auto-resolves when metric returns to normal
 - **Weekly review:** On-call reviews alert volume; tunes noise down
-

13. Environment Configuration

13.1 Configuration Sources

1. **Environment variables** (highest priority) — for secrets + deployment-specific
2. **.env file** (local dev only) — gitignored
3. **Database** — feature flags, per-tenant config
4. **Default values** (lowest priority) — in code

13.2 Environment Variable Catalog

Variable	Required	Description	Example
<code>NODE_ENV</code>	Yes	<code>development</code> / <code>staging</code> / <code>production</code>	<code>production</code>
<code>PORT</code>	Yes	Backend port	<code>3030</code>
<code>DATABASE_URL</code>	Yes	Postgres connection string	<code>postgresql://...</code>
<code>DATABASE_POOL_SIZE</code>	No	Prisma pool size	<code>10</code>
<code>REDIS_URL</code>	Yes	Redis connection string	<code>redis://...</code>
<code>JWT_SECRET</code>	Yes	JWT signing key	(32+ chars random)
<code>JWT_ISSUER</code>	No	JWT issuer	<code>suop-erp</code>
<code>S3_ENDPOINT</code>	Yes	S3 / MinIO endpoint	<code>https://s3.ap-south-1.amazonaws.com/...</code>

Variable	Required	Description	Example
<code>S3_BUCKET</code>	Yes	S3 bucket name	<code>suop-prod</code>
<code>S3_ACCESS_KEY</code>	Yes	S3 access key	<code>AKIA...</code>
<code>S3_SECRET_KEY</code>	Yes	S3 secret key	(secret)
<code>SMTP_HOST</code>	Yes	SMTP server	<code>smtp.sendgrid.net</code>
<code>SMTP_PORT</code>	Yes	SMTP port	<code>587</code>
<code>SMTP_USER</code>	Yes	SMTP username	<code>apikey</code>
<code>SMTP_PASS</code>	Yes	SMTP password	(secret)
<code>SENTRY_DSN</code>	No	Sentry DSN	<code>https://...</code>
<code>TWILIO_ACCOUNT_SID</code>	No	Twilio SID	<code>AC...</code>
<code>TWILIO_AUTH_TOKEN</code>	No	Twilio token	(secret)
<code>NEXT_PUBLIC_API_URL</code>	Yes	Frontend API URL	<code>https://api.sudhamrit.com</code>
<code>NEXT_PUBLIC_SUPABASE_URL</code>	No	Supabase URL	<code>https://...supa</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	No	Supabase anon key	<code>eyJ...</code>

13.3 Environment Validation

All env vars validated at boot via zod:

```
const envSchema = z.object({
  NODE_ENV: z.enum(['development', 'staging',
    'production']),
  // ...
})
```

```
} )  
export const env = envSchema.parse(process.env)
```

App refuses to start if validation fails — fail fast.

14. Backup & Recovery

(see Disaster Recovery document for full details)

14.1 DevOps Responsibilities

- Backup scripts in `infra/scripts/backup/`
 - Restore scripts in `infra/scripts/restore/`
 - Daily restore-test job
 - Monthly DR drill
 - Annual full disaster simulation
-

15. Open Questions for Approval

#	Decision	Recommendation	Alternatives
Q-DV1	Container orchestration	AWS ECS Fargate	Kubernetes (EKS)
Q-DV2	Blue-green vs canary	Blue-green	Canary
Q-DV3	Deploy frequency	Twice weekly	Daily, weekly
Q-DV4	IaC tool	Terraform	Pulumi, CloudFormation
Q-DV5	Monitoring stack	Prometheus + Grafana + Loki + Tempo	Datadog (all-in-one)

#	Decision	Recommendation	Alternatives
Q-DV6	Error tracking	Sentry	Rollbar, Datadog
Q-DV7	Alert routing	PagerDuty + Slack	OpsGenie
Q-DV8	Image registry	AWS ECR	Docker Hub, GitHub Container Registry
Q-DV9	CI/CD platform	GitHub Actions	GitLab CI, CircleCI
Q-DV10	Mobile deploy	EAS (Expo)	Fastlane + manual

Approval Block

Approved by: _____ **Date:** _____

End of Enterprise DevOps Architecture Document